DATABASE PORTFOLIO: PROJECT 1

NEXUS-GLOBAL FASHION & LOGISTICS OPERATIONS SYSTEM

1. The Scene & Business Objective

The Scenario: A growing fashion boutique imports retail items, custom dresses, and raw textiles from international suppliers (in Chinese Yuan - CNY) and sells them locally in Nigeria across different states (in Naira - NGN).

The Operational Challenges:

- **Freight Costs:** Shipping companies charge fees based on physical space occupied (Cubic Meters / CBM) rather than weight alone. Large, light objects can be more expensive to transport than small, heavy objects.
- **Currency Exchange Rates:** Product purchasing prices fluctuate based on daily CNY-to-NGN conversion rates.
- **Sales Monitoring:** Management requires a dynamic way to track and filter high-value orders across regional hubs (Lagos, Abuja, Port Harcourt, etc.) to optimize targeted marketing.

Objective: To build a relational database architecture from scratch that solves these entry bottlenecks by automating landing cost mathematics, creating unified views, and indexing tables for high-velocity lookups.

2. Simplified Tasks, Queries, and Technical Outputs

Task 1: The Container Space Audit (Physical Volume Calculation)

Goal: Calculate the exact physical space occupied in Cubic Meters (CBM) for every single inventory item to accurately forecast freight requirements.

The Logistics Formula Applied:

$$CBM = (Length\_CM \times Width\_CM \times Height\_CM) / 1{,}000{,}000$$

Query Implemented:

```
WITH ProductVolume_CTE AS (
    SELECT ProductID, ProductName, Category,
           (Length_CM * Width_CM * Height_CM / 1000000) AS Volume_CBM
    FROM Product_Catalog AS PC
)
SELECT * FROM ProductVolume_CTE
ORDER BY Volume_CBM DESC;
```

Output Dataset (Sample Rows):

ProductID	ProductName	Category	Volume_CBM
PC029	Raw Denim Textile Bolt	Textiles	0.109760000000
PC011	Brocade Fabric Bolt	Textiles	0.108000000000
PC012	Egyptian Cotton Roll	Textiles	0.093750000000
PC025	Pure Linen Fabric Roll	Textiles	0.062920000000
PC014	Chiffon Sheer Fabric	Textiles	0.040000000000
PC023	Travel Backpack Waterproof	Accessories	0.027648000000
...	...	...	...
PC010	Silk Neck Scarf	Accessories	0.000225000000

Task 2: Automatic Landing Cost Calculator

Goal: Eliminate manual spreadsheet calculation errors. Build an automated engine where staff input a product code and the current market exchange rate to instantly determine the true cost of bringing that item into the Nigerian warehouse.

The Financial Logic Applied:

- Base Cost (NGN) = Foreign Unit Cost (CNY) × Market Exchange Rate
- Freight Cost (NGN) = Box Volume (CBM) × ₦250,000 (Flat Shipping Rate)
- Total Landing Cost = Base Cost + Freight Cost

Query Implemented:

```
CREATE PROCEDURE sp_CalculateProductLandingCost
    @TargetProductID VARCHAR(10),
    @ExchangeRate DECIMAL(10,2)
AS
SELECT
    ProductID,
    ProductName,
    (Unit_Cost_CNY * @ExchangeRate) AS Base_Cost_NGN,
    ((Length_CM * Width_CM * Height_CM / 1000000) * 250000) AS
Freight_Cost_NGN,
    ((Unit_Cost_CNY * @ExchangeRate) + ((Length_CM * Width_CM * Height_CM / 1000000)
* 250000)) AS Total_Landing_Cost_NGN
FROM Product_Catalog
WHERE ProductID = @TargetProductID;
GO
```

Testing the Engine:

```
EXEC sp_CalculateProductLandingCost @TargetProductID = 'PC002', @ExchangeRate = 210;
EXEC sp_CalculateProductLandingCost @TargetProductID = 'PC029', @ExchangeRate = 210;
```

Output Datasets:

Scenario A: The Denim Jacket (PC002)

ProductID	ProductName	Base_Cost_NGN	Freight_Cost_NGN	Total_Landing_Cost_NGN
PC002	Oversized Denim Jacket	38,850.00	3,150.00	**42,000.00**

Scenario B: The Raw Denim Textile Bolt (PC029)

ProductID	ProductName	Base_Cost_NGN	Freight_Cost_NGN	Total_Landing_Cost_NGN
PC029	Raw Denim Textile Bolt	205,800.00	27,440.00	**233,240.00**

Task 3: High-Value Sales Audit Dashboard

Goal: Identify high-impact local revenue transactions. The system must automatically look at all historic purchases and isolate transactions where the amount paid is strictly higher than the current national average company-wide order value.

Query Implemented:

```
-- Step A: Simplify the table connections using a Virtual View
CREATE VIEW v_MasterSales AS
SELECT
    SO.OrderID,
    SO.Customer_Location,
    SO.Amount_Paid_NGN,
    SO.Sale_Date,
    PC.ProductName,
    PC.Category
FROM Sales_Orders AS SO
INNER JOIN Product_Catalog AS PC ON SO.ProductID = PC.ProductID;
GO

-- Step B: Run a dynamic subquery filter against the View
SELECT * FROM v_MasterSales
WHERE Amount_Paid_NGN > (SELECT AVG(Amount_Paid_NGN) FROM v_MasterSales)
ORDER BY Amount_Paid_NGN DESC;
```

Output Dataset:

OrderID	Customer_Location	Amount_Paid_NGN	Sale_Date	ProductName	Category
5017	Lagos	240000.00	2026-02-25	Winter Trench Coat	Clothing
5037	Port Harcourt	240000.00	2026-04-29	Winter Trench Coat	Clothing
5020	Lagos	195000.00	2026-03-05	Velvet Evening Gown	Clothing
5022	Port Harcourt	180000.00	2026-03-10	Stainless Steel Quartz Watch	Accessories
5004	Abuja	180000.00	2026-01-15	Stainless Steel Quartz Watch	Accessories
5039	Lagos	180000.00	2026-05-05	Stainless Steel Quartz Watch	Accessories
5027	Abuja	165000.00	2026-03-26	Cashmere Sweater	Clothing
5025	Kano	125000.00	2026-03-19	Leather Chelsea Boots	Shoes
5006	Lagos	125000.00	2026-01-20	Leather Chelsea Boots	Shoes
5013	Lagos	110000.00	2026-02-10	Strappy Stiletto Heels	Shoes
5034	Abuja	110000.00	2026-04-19	Strappy Stiletto Heels	Shoes
5024	Abuja	98000.00	2026-03-15	Merino Wool Yarn Pack	Textiles
5016	Lagos	95000.00	2026-02-22	Running Spikes Pro	Shoes
5026	Lagos	90000.00	2026-03-22	Suede Loafers	Shoes

Task 4: System Performance Architecture

Goal: Ensure rapid page loads as database records scale from hundreds to millions of entries. Prevent expensive "Table Scans" when regional dashboards request data sorted by city and date.

Query Implemented:

```
CREATE INDEX IX_Sales_Location_Date
ON Sales_Orders (Customer_Location, Sale_Date);
```

Note: This index works silently in the background, transforming costly lookups into instant index searches.

3. Conclusion & Core Competencies Shown

This project successfully demonstrates professional-level competency across major database management criteria:

- **Relational Architecture:** Enforcing explicit relationships via Primary Keys, auto-incrementing Identities, and Foreign Keys to preserve data integrity and prevent broken inputs.
- **Mathematical & Financial Engine Design:** Performing inline calculations across changing fields, converting currency models, and manipulating physical dimension constraints within code blocks.
- **Advanced Logic Abstraction:** Using Common Table Expressions (CTEs) to simplify nested logic, Views to simplify multi-table joins for operational stakeholders, and Stored Procedures to bundle reusable business calculators.
- **Performance Tuning:** Designing multi-column composite non-clustered indexes to prepare operations databases for high-volume enterprise expansion.